

1. Introduction

I build an instance segmentation model for four cell types in colored H&E-stained pathology images. My method is a Mask R-CNN with a ResNet-50 FPN-v2 backbone, adapted to this dataset along three axes:

- (1) Anchors tuned for small objects. I replace the default anchor sizes (32, 64, 128, 256, 512) with (8, 16, 32, 64, 128) and re-initialize the RPN head, since cell nuclei in this dataset are typically only 8–40 pixels across.
- (2) All detection heads trained from scratch. I load only ImageNet-1K weights into the ResNet-50 backbone and set weights=None, so the FPN neck, RPN head, box head, and mask head are all randomly initialized.
- (3) Stabilization for a small dataset. With only 209 training images I use heavy geometric and photometric augmentation, multi-scale training (short edge $\in \{640, 800\}$), weight EMA (decay = 0.9998), AMP mixed precision, and a two-LR AdamW optimizer with linear warmup.

I initialize the model with weights=None (no COCO pretraining) and weights_backbone=ResNet50_Weights.IMAGENET1K_V2, so only the ResNet-50 backbone is pretrained on ImageNet-1K while the FPN, RPN, box head, and mask head are all trained from scratch.

```
model = maskrcnn_resnet50_fpn_v2(  
    weights=None,  
    weights_backbone=ResNet50_Weights.IMAGENET1K_V2,
```

The model reaches validation AP50 = 0.5231 and a test AP50 = 0.5346 on the Kaggle leaderboard.

Github link: <https://github.com/mses0424/Visual-Recognitionusing-Deep-Learning-hw3/tree/main>

2. Method

2.1 Data Preprocessing

Per-sample loading

There are four classes of image, my loader reads image.tif with tiff file. If the image is uint16 (some scanners produce 16-bit TIFFs), rescale to uint8 by max-normalization.

For each present classX.tif, enumerates np.unique values, builds a binary mask per instance ID, and any instance consisting of fewer than 4 pixels is discarded as noise (degenerate annotations).

For all valid instances across the four classes, the loader extracts coordinates by computing axis-aligned bounding boxes, which are stored in xyxy format (denoting the top-left and bottom-right corners) and paired with their respective labels from Class 1 to Class 4.

Train/validation split

Uses a deterministic random.shuffle (seed = 42) and reserve 10% of training samples (≈ 20 images) for validation. The validation set is held out throughout training and is only used to track AP50 and to select the best checkpoint.

Training Pipeline

Uses augmentations to jointly transform images, masks, and boxes

(1) Geometric Transforms:

Horizontal/vertical flips and random 90° rotations (50%).

Affine: Random scaling (85%~115%), translation ($\pm 8\%$), and rotation ($\pm 25^\circ$).

(2) Color & Noise (Addressing stain variations):

CLAHE: Normalizes staining intensities. CLAHE(clip_limit=[1, 3], p=0.3)

Color Jitter: Randomly adjusts hue, saturation, or brightness to combat color shifts across different slides.

Gaussian Noise: Improves robustness against sensor noise.

(3) Post-processing:

Recompute Boxes: Bounding boxes are regenerated from transformed masks to

prevent distortion caused by affine warps.

Filtering: Drops instances if the post-augmentation mask is smaller than 2 pixels in either axis.

Standardization: Applies ImageNet normalization and converts to Tensor.

For validation and test pipeline no augmentation used, only applies normalization and Tensor conversion to maintain the original data properties.

2.2 Model Architecture

Mask R-CNN ResNet-50 FPN v2 with custom anchors and re-initialized heads.

Backbone — ResNet-50

Loads the `ResNet50_Weights.IMAGENET1K_V2` weights, which were trained on ImageNet-1K classification only (no detection or segmentation pretraining).

Following torchvision's `_resnet_fpn_extractor`, I set `trainable_backbone_layers=4`, which keeps the stem (conv1 + bn1) frozen and unfreezes stages 2–5. The reason is the early stem learns generic low-level edge filters that transfer well, while the deeper stages benefit from adapting to the H&E color distribution.

Neck — Feature Pyramid Network (FPN)

Mask R-CNN v2 uses the FPN-v2 implementation, which differs from the original FPN by adding a 3×3 convolution after each lateral connection (instead of just element-wise addition). This produces five feature maps P2–P6 at strides 4, 8, 16, 32, 64 with 256 channels each.

Region Proposal Network (RPN)

The default torchvision anchor generator uses sizes (32, 64, 128, 256, 512) over the five FPN levels. Cell nuclei in dataset are typically only 8–40 pixel in their longest dimension, so replaces these with (8, 16, 32, 64, 128). Then re-instantiate the RPN head (`model.rpn.head = RPNHead(out_channels, num_anchors_per_location)`) so that its convolutional weights are randomly re-initialized.

Region of Interest Alignment (RoIAlign) Heads — Box + Mask

For each proposal, RoIAlign extracts a $7 \times 7 \times 256$ feature for the box branch and a $14 \times 14 \times 256$ feature for the mask branch. The v2 architecture uses 4 conv layers (plus one 3×3 conv) in the mask head, replacing the original 4-conv design. Both predictors are constructed for `num_classes = 5` (1 background + 4 cell types) and are fully randomly initialized without using COCO pretraining.

Since a single image often contains more than 100 cells, I raised the maximum detection limit to 300 per image and relaxed the candidate box limits to prevent missing targets in dense scenes and the full model has ≈ 46 M parameters.

2.3 Hyperparameter Settings

Optimizer

Two AdamW groups: Backbone group learning rate is $5e-5$. Heads group (RPN + box + mask) learning rate is $2e-4$. Weight decay is $1e-4$ for both groups.

Schedule

3-epoch linear warmup (`start_factor=0.01`), cosine annealing over the remaining 47 epochs, for a total of 50 epochs. Mixed precision (`torch.cuda.amp`) and clip gradients at norm 5.0.

Multi-scale training

Randomly resizes the short edge to 640 or 800 pixels per batch (max long edge 1333) to consistently improve small-object detection.

Exponential Moving Average (EMA)

New EMA weights = $(0.9998 \times \text{Old EMA weights}) + (0.0002 \times \text{Current model weights})$. At each evaluation run COCOEval twice, once with the live weights and once with the EMA copy and save whichever has the higher AP50 as the best.

Region Proposal Network (RPN)

`pre_nms_top_n_train = 1000`. Determines the maximum number of region proposals to retain based on their scores. Prior to applying Non-Maximum Suppression (NMS).

`post_nms_top_n_train = 1000`. Defines the final number of top-scoring proposals to keep after NMS has eliminated redundant, highly overlapping bounding boxes.

Above is parameter of training, when test is 1000(pre) and 500(post).

Inference Settings

Input images are resized, short edge is 800 pixels, and maximum long edge is 1024 pixels. The confidence score threshold is 0.05 and maximum number of detections per image is 500.

3. Result

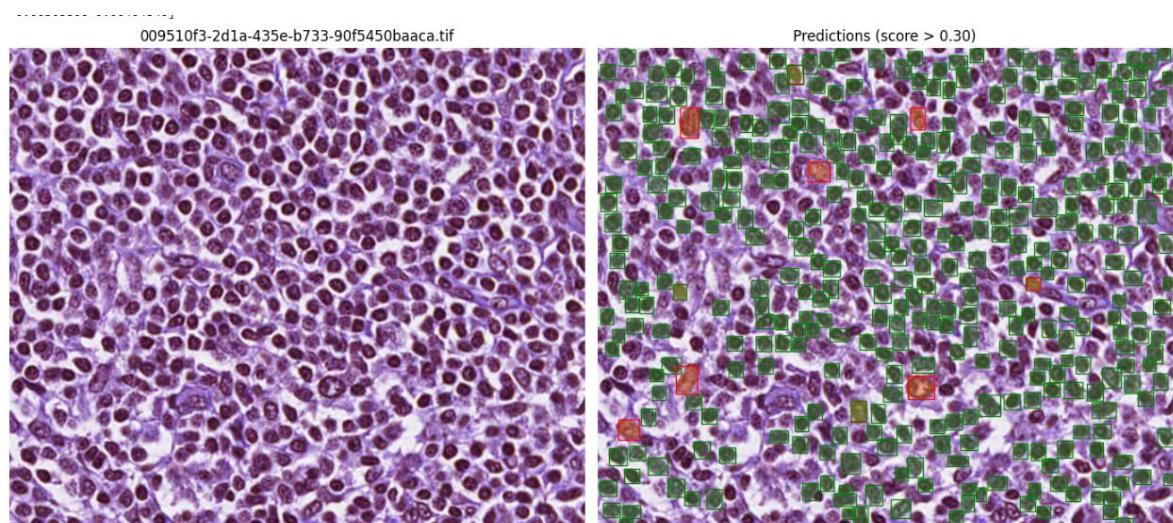
3.1 output picture

best epoch 49 result

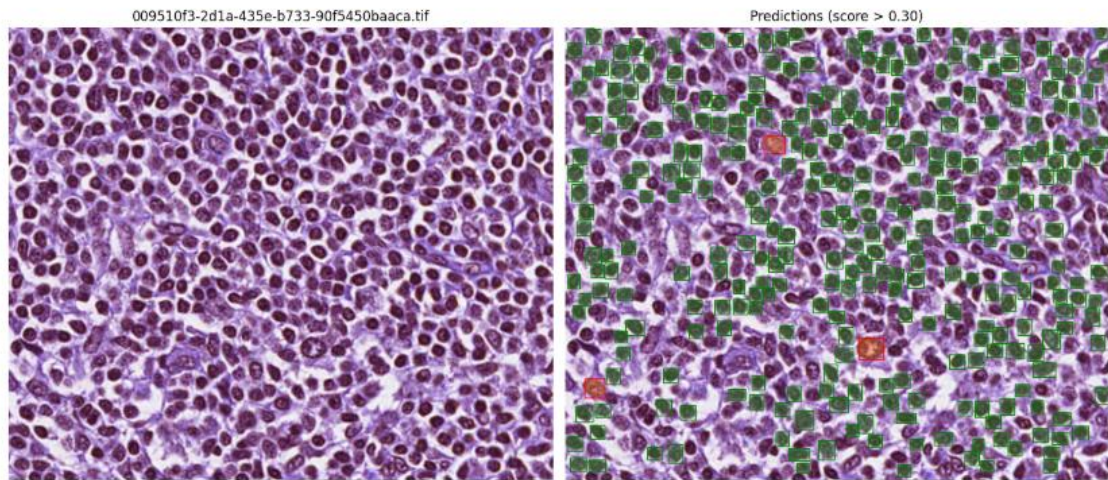
```
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.343
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.523
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.416
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.285
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.286
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = -1.000
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.048
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.266
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.429
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.334
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.402
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = -1.000
[Ep 49/50] loss=0.942 plain AP50=0.5203 ema AP50=0.5231 (1024s)
```

Example test output (detection=500)

more green and red box than detection=300

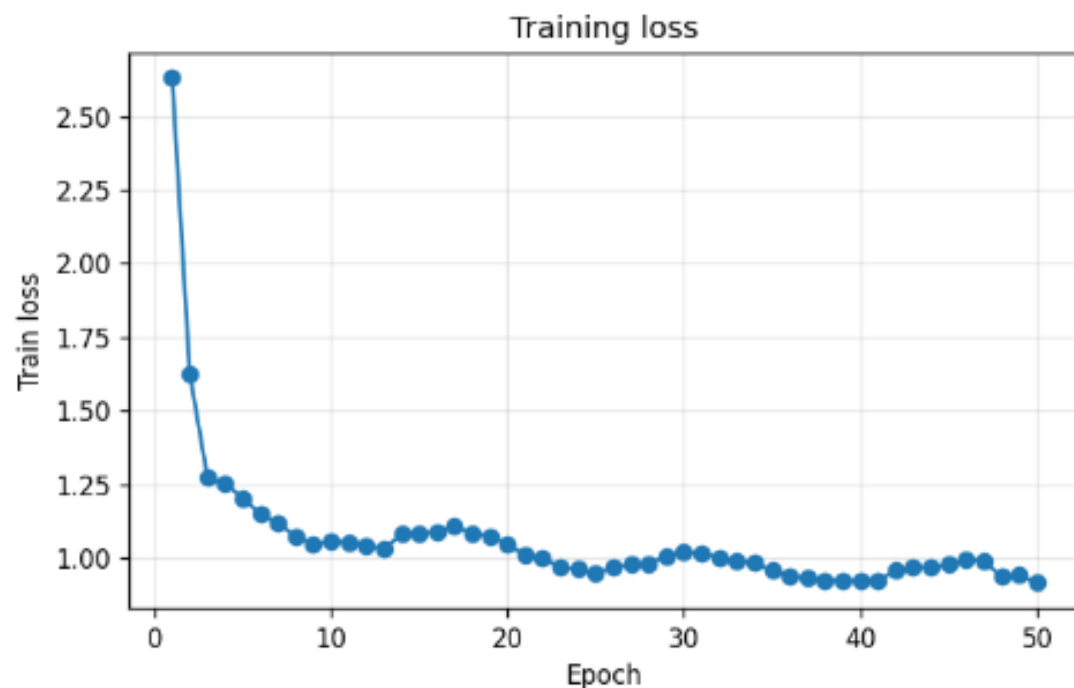


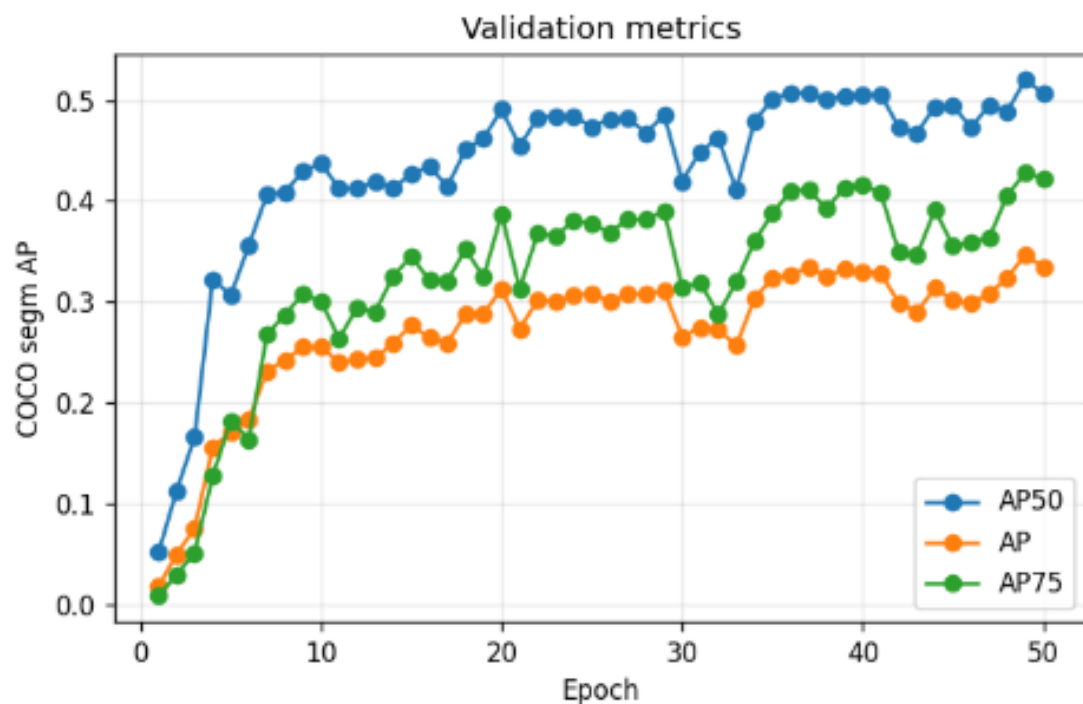
detection=300



Based on the output images, the model successfully detects only about half of the actual cells in the field of view. I adjusted the confidence threshold to verify if the remaining cells were predicted with lower scores, but the visual results remained entirely unchanged. So the problem is model miss these cells. I adjust the number of max_detection from 300 to 500. The result show that 500 is better. When I adjust to 1000, there are some skips happen, so I use 500 as my final output. (mAP50 of 300 is 0.5146, mAP50 of 500 is 0.5346 from kaggle).

3.2 Training Curve





Training loss drops rapidly in the first 5 epoch and stabilizes near 0.9. No obvious signs of overfitting are observed. The Validation AP50 shows an overall upward trajectory. Although a noticeable drop occurs between epochs 31–35, the performance successfully stabilizes and resumes its ascent, reaching a peak of approximately 0.52 by the 49th epoch.

4. Additional Experiments

4.1 Weight Exponential Moving Average (EMA)

Hypothesis

With only 209 training images, mini-batch gradients are noisy and per-step weights oscillate. A slow EMA (decay = 0.9998) of the weights should provide a more stable evaluation target than the live weights.

Why it may (or may not) work

EMA averages out high-frequency oscillations, so it should help during AP troughs caused by stochastic batch composition. However, when the model is monotonically improving, EMA simply lags the live weights.

Results

epoch	AP50 (live)	AP50 (EMA)
31	0.4489	0.4620
33	0.4109	0.4353
34	0.4790	0.4785
35	0.5011	0.4989
42	0.4729	0.4957
43	0.4673	0.4887
47	0.4949	0.4904
49	0.5203	0.5231

EMA wins more often than it loses, and the trade-off is highly asymmetric: when EMA wins, the gain can be large (like epoch33, +0.024); when it loses, the gap is only a few thousandths (≤ 0.005). With $\max(\text{live}, \text{EMA})$ selection, EMA can only help and never hurt. The results show that this method is effective.

4.2 Replacing Box-Head Regression Loss: Smooth-L1 $\rightarrow$ GloU

Hypothesis

Smooth-L1 on the four regression deltas is not directly aligned with the IoU-based AP metric. I replace it with GloU loss [Rezatofighi et al., CVPR 2019] on the box head only, while keeping all other loss terms unchanged.

Why it may (or may not) work

GloU directly optimizes geometric overlap, so it should give a more relevant gradient — particularly at strict IoU thresholds. However, when predicted boxes are far from the GT, the GloU gradient becomes weak, so I keep smooth-L1 for the first 5 epochs as warmup before switching.

Results

I run this method with 30 epochs, best mAP50 is 0.4882 and best mAP75 is 0.3912. Only smooth-L1 the best mAP50 is 0.4914 and best mAP75 is 0.3976. The result shows this method doesn't work. Likely because cell nuclei are small and roughly isotropic, smooth-L1 is already near-optimal for this dataset, leaving little room for GloU to exploit. I therefore didn't use GloU in my final code.

5. References

- [1] K. He, G. Gkioxari, P. Dollár, R. Girshick. "Mask R-CNN." *ICCV*, 2017.
- [2] Y. Li, S. Xie, X. Chen, P. Dollár, K. He, R. Girshick. "Benchmarking Detection Transfer Learning with Vision Transformers." *arXiv:2111.11429*, 2021. (*Source of the v2 detection recipe used by torchvision.*)
- [3] A. Buslaev *et al.* "Albumentations: Fast and Flexible Image Augmentations." *Information*, 2020.
- [4] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie. "Feature Pyramid Networks for Object Detection." *CVPR*, 2017.
- [5] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, S. Savarese. "Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression." *CVPR*, 2019.
- [6] S. Ren, K. He, R. Girshick, J. Sun. "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks." *NeurIPS*, 2015.